RKNN2_Toolkit_Lite_Python_API_Reference

Document ID: RK-KF-YF-439

Version: V2.5.0

Date: 2026-09-30

Confidentiality: □Top Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION OR CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY.

THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT PRIOR NOTICE AT ANY TIME DUE TO PRODUCT UPGRADES OR OTHER REASONS.

Trademark Statement

“Rockchip”, “瑞芯微”, and “瑞芯” are registered trademarks of Rockchip and are owned by Rockchip.

All other trademarks or registered trademarks mentioned in this document are owned by their respective owners.

All rights reserved. © Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, no entity or individual may extract, copy, or distribute this document, in whole or in part and in any form, without prior written permission from Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89 Software Boulevard, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer Service Tel: +86-4007-700-590

Customer Service Fax: +86-591-83951833

Customer Service E-mail: fae@rock-chips.com

Preface

Overview

This document is the RKNN Toolkit Lite2 Python API Reference.

RKNN Toolkit Lite2 provides Python programming interfaces for Rockchip NPU platforms and is mainly used to load and run RKNN models on target devices. RKNN models must first be converted using RKNN Toolkit2.

This manual focuses on the public interfaces of `RKNNLite` and describes the applicable environment, basic usage flow, model loading, Runtime initialization, model inference, SDK version query, supported-platform query, and resource release.

Target Audience

This document is primarily intended for the following engineers:

- Technical Support Engineers
- Software Development Engineers

Revision History

Version	Modifier	Date	Description	Reviewer
V1.2.0	Raul	2022-01-13	Initial version.	Vincent
V1.3.0	Raul	2022-04-27	Update version.	Vincent
V1.4.0	Raul	2022-08-30	Update the description of the `core_mask` parameter of `init_runtime`.	Vincent
V1.5.0	Raul	2023-05-18	1. Add RK3562 support. 2. Add Python 3.8 and Python 3.10 support on ARM64 platforms.	Vincent
V1.5.2	Raul	2023-08-21	Update version.	Vincent
V1.6.0	Raul	2023-10-10	1. Add an installation package for AARCH64 Python 3.11. 2. Add dynamic Shape model support to the inference API. 3. Add support for 4-D NCHW-format inputs to the inference API.	Vincent
V1.6.2	Raul	2024-03-14	1. Add RK3576 support. 2. Update the description of the `core_mask` attribute of `init_runtime`.	Vincent
V2.5.0	HPC	2026-09-30	1. Restructure the document. 2. Add descriptions for encrypted model loading. 3. Add the `reuse_outputs` parameter description for the `inference` API.	Vincent

Table of Contents

1. API Reference Overview
    - 1.1 Applicable Version and Platforms
    - 1.2 Installation and Environment Setup
    - 1.3 Basic Usage Flow
    - 1.4 Example Program
    - 1.5 API Return Conventions
2. RKNNLite Programming Model
    - 2.1 Object Lifecycle
    - 2.2 Relationship Between Model, Runtime, and Inference
    - 2.3 Input Data, Data Layout, and Output
        - 2.3.1 inputs
        - 2.3.2 data_type and data_format
        - 2.3.3 inputs_pass_through
        - 2.3.4 get_frame_id
        - 2.3.5 reuse_outputs
        - 2.3.6 Inference Output and Failure Return
    - 2.4 NPU Core Configuration
    - 2.5 Query APIs and Resource Release
3. RKNNLite Class
    - 3.1 Object Initialization and Release
        - init
        - release
    - 3.2 Model Loading and Runtime Initialization
        - load_rknn
        - init_runtime
    - 3.3 Model Inference
        - inference
    - 3.4 Device, Version, and Platform Queries
        - list_devices
        - get_sdk_version
        - list_support_target_platform
4. Error Handling
    - 4.1 Model Loading and Runtime Initialization Errors
    - 4.2 Inference Input and Return-Value Checks

4.3 NPU Core Configuration Checks

4.4 Logging

5. Appendix

5.1 Reference Documents

5.2 Issue Feedback

1. API Reference Overview

This manual describes the Python APIs of RKNN Toolkit Lite2. It focuses on the `RKNNLite` object and covers environment requirements, model loading, Runtime initialization, model inference, SDK version query, and supported-platform query.

RKNN Toolkit Lite2 is used to deploy RKNN models on Rockchip NPU platforms. Model conversion is performed by RKNN Toolkit2 and is outside the scope of this manual.

1.1 Applicable Version and Platforms

This document corresponds to RKNN Toolkit Lite2 V2.5.0.

The currently supported range is:

Item	Supported Range
RKNN Toolkit Lite2 version	V2.5.0
Hardware platforms	RK3562, RK3566 series, RK3568 series, RK3576 series, RK3588 series
Operating system	Debian 10 / 11 / 12 (aarch64)
Python	3.8 / 3.9 / 3.10 / 3.11 / 3.12
Python dependencies	`numpy`, `ruamel.yaml`, `psutil`

1.2 Installation and Environment Setup

RKNN Toolkit Lite2 can be installed with `pip3 install`.

If `python3` or `pip3` is not installed, install the required base packages first:

```
sudo apt-get update
sudo apt-get install -y python3 python3-dev python3-pip gcc
```

Some dependent modules need to compile source code during installation, so `python3-dev` and `gcc` are installed together to avoid build failures.

The examples also use `opencv-python` and `numpy`:

```
sudo apt-get install -y python3-opencv
sudo apt-get install -y python3-numpy
```

Notes:

1. RKNN Toolkit Lite2 itself does not depend on `opencv-python`, but the examples use it to process images.
2. Installing `numpy` directly with `pip3` may fail on Debian 10 firmware. The system-package installation method above is recommended.

The installation packages for each RKNN Toolkit Lite2 platform are located in the SDK `packages` directory. Enter this directory and install the Wheel corresponding to the Python version:

```
# Python 3.8
pip3 install rknn_toolkit_lite2-2.x.0-cp38-cp38-linux_aarch64.whl

# Python 3.9
pip3 install rknn_toolkit_lite2-2.x.0-cp39-cp39-linux_aarch64.whl

# Python 3.10
pip3 install rknn_toolkit_lite2-2.x.0-cp310-cp310-linux_aarch64.whl

# Python 3.11
pip3 install rknn_toolkit_lite2-2.x.0-cp311-cp311-linux_aarch64.whl

# Python 3.12
pip3 install rknn_toolkit_lite2-2.x.0-cp312-cp312-linux_aarch64.whl
```

1.3 Basic Usage Flow

Before using RKNN Toolkit Lite2, use RKNN Toolkit2 to convert models exported from deep learning frameworks into RKNN models.

The basic usage flow is:

```
Create RKNNLite object
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
Prepare and preprocess input data
    ↓
inference()
    ↓
Post-process inference results
    ↓
release()
```

Where:

- `RKNNLite()` creates the RKNN Toolkit Lite2 Python object;
- `load_rknn()` loads an RKNN model;
- `init_runtime()` initializes the Runtime environment;
- `inference()` performs model inference;
- `release()` releases the RKNNLite object resources after use.

Before calling `inference()`, prepare and preprocess the input data, then configure `data_format` and other parameters according to the actual input information. After inference, the outputs normally require post-processing to complete the upper-layer application functionality.

1.4 Example Program

The SDK provides a `resnet18` example in the `examples` directory. The example loads a ResNet18 model, runs inference, and outputs the Top-5 classification results for the test image.

Run it as follows:

1. Prepare a development board with RKNN Toolkit Lite2 installed.
2. Copy the example to the development board.
3. Enter the `examples/resnet18` directory and run:

```
python3 test.py
```

Reference output:

```
-----TOP 5-----
[812]: 0.999442994594574
[404]: 0.0004096269840374589
[657]: 3.284541890025139e-05
[833]: 2.6112385967280716e-05
[895]: 1.8509887013351545e-05
```

1.5 API Return Conventions

According to the current `RKNNLite` Python implementation, the main APIs use the following return conventions:

API	Normal Return	Failure / Special Return
`load_rknn()`	`0`	`-1`
`list_devices()`	`(adb_devices, ntb_devices)`, where both items are `list` objects	No separate failure return is defined
`init_runtime()`	`0`	Returns `-1` if parameter validation, model loading, Runtime creation, or initialization fails
`inference()`	`list`; each item is an output `ndarray`. If `get_frame_id=True`, the Frame ID is appended to the end of the returned list	Returns `None` if the Runtime is not initialized, or if setting inputs, running the model, or retrieving outputs fails
`get_sdk_version()`	SDK version information string	Returns `None` if the Runtime is not initialized or the query fails
`list_support_target_platform()`	`dict` returned by the platform query function	Returns `None` if the specified RKNN model path does not exist
`release()`	`None`	`None`

For APIs that return status codes, explicitly check the return value. For example:

```
ret = rknn_lite.init_runtime(core_mask=RKNNLite.NPU_CORE_AUTO)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

For `inference()`, `get_sdk_version()`, and `list_support_target_platform()`, also check whether the return value is `None` to avoid processing invalid results when prerequisites are not met or an internal operation fails.

2. RKNNLite Programming Model

`RKNNLite` is the main Python entry point of RKNN Toolkit Lite2. Applications use this object to load RKNN models, initialize the Runtime, run inference, query version information, and release resources.

RKNN Toolkit Lite2 is intended for device-side deployment. A deep learning framework model is first converted into an RKNN model with RKNN Toolkit2, then the device-side application runs it through `RKNNLite()` → `load_rknn()` → `init_runtime()` → `inference()` → `release()`.

Understanding the relationship among the RKNN model, Runtime, input data, and NPU Core configuration helps when using the APIs described in Chapter 3.

2.1 Object Lifecycle

The typical RKNN Toolkit Lite2 lifecycle is:

```
Create RKNNLite object
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
Prepare and preprocess input data
    ↓
inference()
    ↓
Post-process inference results
    ↓
release()
```

Where:

- `RKNNLite()` creates an RKNNLite object and can enable detailed logging through `verbose` and `verbose_file`;
- `load_rknn()` loads the RKNN model from the specified path;
- `init_runtime()` initializes the Runtime environment required for model execution;
- `inference()` executes inference on prepared inputs and returns the outputs;
- `release()` releases resources associated with the RKNNLite object.

`init_runtime()` checks whether a model has been loaded with `load_rknn()`, and `inference()` checks whether the Runtime has been initialized. Therefore, the normal call order is `load_rknn()` → `init_runtime()` → `inference()`.

A typical application structure is:

```
rknn_lite = RKNNLite(verbose=True)

ret = rknn_lite.load_rknn('./model.rknn')
if ret != 0:
    print('Load RKNN model failed')
    exit(ret)

ret = rknn_lite.init_runtime(
    core_mask=RKNNLite.NPU_CORE_AUTO
)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)

outputs = rknn_lite.inference(inputs=[input_data])

# Post-process outputs according to the model type.
# ...

rknn_lite.release()
```

Input data should be preprocessed before `inference()` is called, and `data_format` should be configured according to the input dimensions and the actual data layout. After inference, the outputs normally require post-processing to implement the upper-layer application logic.

2.2 Relationship Between Model, Runtime, and Inference

The deployment process can be divided into the following stages:

- **Model conversion**: use **RKNN Toolkit2** to convert a deep learning framework model into an **RKNN model**;
- **Model loading**: use `load_rknn()` to load the **RKNN model** into the `RKNNLite` object;
- **Runtime initialization**: use `init_runtime()` to initialize the **Runtime environment**;
- **Inference execution**: use `inference()` to obtain **inference results**.

RKNN Toolkit2 and RKNN Toolkit Lite2 serve different stages:

Stage	Tool / API	Purpose
Model conversion	RKNN Toolkit2	Convert a deep learning framework model into an RKNN model
Model loading	`load_rknn()`	Load an RKNN model for device-side execution
Runtime initialization	`init_runtime()`	Initialize the Runtime environment
Model execution	`inference()`	Run inference and return outputs
Resource release	`release()`	Release RKNNLite object resources after use

Besides the inference API, RKNN Toolkit Lite2 also provides version and platform query APIs:

```
load_rknn()
    ↓
init_runtime()
    ↓
get_sdk_version()
```

`get_sdk_version()` can only be called after the Runtime has been initialized successfully. If the Runtime has not been initialized, the API returns `None`. Because `init_runtime()` itself requires a model to have been loaded, the normal flow is still to call `load_rknn()` first, then `init_runtime()`, and finally `get_sdk_version()`.

`list_support_target_platform()` queries which chip platforms a specified RKNN model can run on. It receives an RKNN model path through the `rknn_model` parameter. If no model path is specified, the API prints the chip platforms currently supported by RKNN Toolkit Lite2 by category.

2.3 Input Data, Data Layout, and Output

The current `inference()` interface signature is:

```
inference(
    inputs,
    data_type=None,
    data_format=None,
    inputs_pass_through=None,
    get_frame_id=False,
    reuse_outputs=False
)
```

`inputs` is the model input list. The other parameters provide the input data type, data layout, Pass Through settings, Frame ID behavior for asynchronous mode, and output memory reuse configuration.

2.3.1 `inputs`

`inputs` is used to pass model input data. Its type is `list`, and each list item is an `ndarray`. Typical usage:

```
outputs = rknn_lite.inference(inputs=[img])
```

For a multi-input model:

```
outputs = rknn_lite.inference(
    inputs=[input0, input1]
)
```

Applications should complete the required preprocessing before calling the inference API.

2.3.2 `data_type` and `data_format`

`data_type` specifies the input data type. The types listed by the current interface include `int8`, `uint8`, `int16`, `float16`, and `float32`. The default value is `None`.

`data_format` specifies the input data layout. The currently supported values are `"nhwc"` and `"nchw"`. The default value is `None`. This parameter is passed directly to the Runtime input-setting logic, so the application must ensure that the value matches the actual input buffer layout.

2.3.3 `inputs_pass_through`

`inputs_pass_through` sets the Pass Through flag for each input. The interface uses `0` / `1` to indicate disabled / enabled, and the values are provided as a list corresponding to the model inputs. The default value is `None`.

2.3.4 `get_frame_id`

`get_frame_id` defaults to `False`. When it is `False`, the API returns the output `ndarray` list directly. When it is `True`, the Runtime Frame ID is appended to the end of the output list. This is mainly used in asynchronous mode when input and output frames need to be associated.

2.3.5 `reuse_outputs`

`reuse_outputs` controls how output array memory is reused. The default value is `False`, and this parameter is an optional optimization that does not change the default behavior.

When it is `False` (default), each output `ndarray` returned by `inference()` owns its own memory and remains valid across multiple calls, conforming to the historical API contract. Applications may safely hold the returned results.

When it is `True`, the API directly returns views of the internal reusable staging buffers, so no large memory allocation occurs per call. This is suitable for long-running inference loops where the outputs are fully consumed before the next inference. **Note**: the returned arrays will be overwritten by the next `inference()` or `get_outputs()` call. If the results must be retained, copy them before the next call (e.g., `np.array(output)`).

2.3.6 Inference Output and Failure Return

Normally, `inference()` returns a list of output `ndarray` objects:

```
Input ndarray list
        ↓
    inference()
        ↓
Output ndarray list
        ↓
Application post-processing
```

If the Runtime has not been initialized, or if an exception occurs while setting inputs, running the model, or retrieving outputs, the API returns `None`. Therefore, application code should check the return value before post-processing.

2.4 NPU Core Configuration

`init_runtime()` uses the `core_mask` parameter to configure the NPU Core working mode. The default value is `RKNNLite.NPU_CORE_AUTO`.

`RKNNLite` currently defines the following Core Mask constants:

`core_mask`	Value	Description
`RKNNLite.NPU_CORE_AUTO`	`0`	Automatic mode. Default value.
`RKNNLite.NPU_CORE_0`	`1`	Core 0 mode.
`RKNNLite.NPU_CORE_1`	`2`	Core 1 mode.
`RKNNLite.NPU_CORE_2`	`4`	Core 2 mode.
`RKNNLite.NPU_CORE_0_1`	`3`	Core 0/1 combined mode.
`RKNNLite.NPU_CORE_0_1_2`	`7`	Core 0/1/2 combined mode. The current interface description marks this as supported only on RK3588.
`RKNNLite.NPU_CORE_ALL`	`0xffff`	All NPU Cores.

The source code notes that `core_mask` is valid only for RK3576 / RK3588. The Core combinations actually supported may vary by chip and should follow Runtime validation. If manual Core selection is not required, use the default `RKNNLite.NPU_CORE_AUTO`.

Typical initialization:

```
ret = rknn_lite.init_runtime(
    core_mask=RKNNLite.NPU_CORE_AUTO
)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

2.5 Query APIs and Resource Release

In addition to model inference, RKNN Toolkit Lite2 provides device enumeration, SDK version query, model supported-platform query, and resource release APIs. Different APIs have different prerequisites.

SDK Version Query

`get_sdk_version()` is used to obtain:

- Runtime version;
- driver version;
- RKNN model version information.

The call sequence is:

```
load_rknn()
    ↓
init_runtime()
    ↓
get_sdk_version()
```

Example:

```
sdk_version = rknn_lite.get_sdk_version()
```

The Runtime must be initialized successfully before calling this API; otherwise, the API returns `None`.

Model Supported-Platform Query

`list_support_target_platform()` queries the chip platforms on which a given RKNN model can run:

```
rknn_lite.list_support_target_platform(
    rknn_model='mobilenet_v1.rknn'
)
```

If `rknn_model=None`, the argument is passed directly to the platform query function, which lists the platforms currently supported by RKNN Toolkit Lite2. If an explicitly specified model path does not exist, including an empty-string path, the API returns `None`.

Resource Release

After finishing operations on an RKNNLite object, call:

```
rknn_lite.release()
```

In the complete deployment flow, resource release comes after model inference and related query operations:

```
RKNNLite()
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
inference() / get_sdk_version()
    ↓
release()
```

`release()` is the final step in the RKNNLite object lifecycle. Release the related resources promptly after model execution on the object is complete.

3. RKNNLite Class

`RKNNLite` is the main Python class in RKNN Toolkit Lite2. It provides object initialization, RKNN model loading, device enumeration, Runtime initialization, model inference, SDK version query, model supported-platform query, and resource release.

3.1 Object Initialization and Release

init

API	**`RKNNLite(...)`**
Function	Initialize an RKNNLite object. Detailed logging can be controlled through `verbose` and `verbose_file`.
Parameters	`verbose`: Specifies whether detailed log information is printed to the screen.
	`verbose_file`: Specifies the log file path. When this parameter is set and `verbose=True`, logs are also written to the specified file.
Return Value	`RKNNLite` object.

Example:

```
# Show detailed logs on screen and save them to inference.log.
rknn_lite = RKNNLite(
    verbose=True,
    verbose_file='./inference.log'
)

# Only show detailed logs on screen.
rknn_lite = RKNNLite(verbose=True)
```

release

API	**`release`**
Function	Release resources used by the RKNNLite object after use.
Parameters	None
Return Value	`None`.

Example:

```
rknn_lite.release()
```

Usage Notes:

- Call `release()` after the RKNNLite object is no longer needed.

- Finish model inference and related query operations before releasing the object.

3.2 Model Loading and Runtime Initialization

load_rknn

API	`load_rknn`
Function	Load an RKNN model.
Parameters	`path`: Path of the RKNN model file.
	`key_path`: (Optional) Path of the key-envelope file for an encrypted model. This parameter is required when loading an RKCE-encrypted model and can be omitted for a plaintext model.
Return Value	Success: `0`; failure: `-1`.

Example (plaintext model):

```
# Load resnet_18.rknn from the current directory.
ret = rknn_lite.load_rknn('./resnet_18.rknn')
```

Example (encrypted model):

```
# Load an encrypted model together with its key envelope.
ret = rknn_lite.load_rknn(
    './encrypted_model.rknn',
    key_path='./encrypted_model_key.bin'
)
```

Usage Notes:

- Call `load_rknn()` before `init_runtime()`.
- The model file should be an RKNN model generated by RKNN Toolkit2.

Encrypted Model Description:

RKNN Toolkit Lite2 supports loading encrypted RKNN models. Encrypted models use a hybrid encryption scheme (AES-256-CTR + RSA-OAEP): an RKCE identifier is written into the model file header, the model body is encrypted with AES-CTR, and the AES key is wrapped by RSA-OAEP into a key envelope (RKKE format) that is deployed to the device together with the encrypted model.

When loading an encrypted model, pass the key-envelope file through the `key_path` parameter. At Runtime, `rknn_init()` automatically detects the RKCE identifier and transparently performs decryption. After decryption, the subsequent inference flow is the same as for a plaintext model.

Encrypted models are generated on a PC/server using the encryption tool provided by RKNN Toolkit2 (the RK public key `rk_public_key.pem` is required). Encryption produces two companion files:

File	Description
`xxx.rknn`	Encrypted RKNN model (RKCE identifier + AES-CTR-encrypted body)
`xxx_key.bin`	RSA-OAEP-encrypted key envelope (deployed together with the encrypted model)

Encrypted models are supported on the RK3562, RK3566 series, RK3568 series, RK3576 series, RK3588 series, RV1126B, RV1103B series, and RV1106B series.

Notes:

- The encrypted model and its key envelope must be used as a matched pair; neither can be omitted.
- The key envelope contains key material required to decrypt the model and should be properly protected from disclosure.
- For more details about the encryption scheme and encryption tool, see the “Model Encryption” section in *RKNN2 SDK Development Guide*.

init_runtime

API	`init_runtime`
Function	Initialize the Runtime environment. This API must be called before inference.
Parameters	`target`: Target platform. Default: `None`. When using the local NPU on an aarch64 target device, this is normally left as `None`.
	`device_id`: Device ID. Default: `None`. Use this only when the current runtime environment explicitly supports device selection.
	`async_mode`: Whether to enable asynchronous mode. Default: `False`.
	`core_mask`: NPU Core working mode. Default: `RKNNLite.NPU_CORE_AUTO`.
Return Value	Returns `0` on success; returns `-1` if prerequisites are not met or Runtime initialization fails.

`core_mask` can use the following constants defined by the `RKNNLite` class:

`core_mask`	Value	Description
`RKNNLite.NPU_CORE_AUTO`	`0`	Automatic mode. Default value.
`RKNNLite.NPU_CORE_0`	`1`	Core 0 mode.
`RKNNLite.NPU_CORE_1`	`2`	Core 1 mode.
`RKNNLite.NPU_CORE_2`	`4`	Core 2 mode.
`RKNNLite.NPU_CORE_0_1`	`3`	Core 0/1 combined mode.
`RKNNLite.NPU_CORE_0_1_2`	`7`	Core 0/1/2 combined mode. The current interface description marks this as supported only on RK3588.

core_mask	Value	Description
RKNNLite.NPU_CORE_ALL	0xffff	All NPU Cores.

The source code notes that core_mask is valid only for RK3576 / RK3588.

Example:

```
ret = rknn_lite.init_runtime(
    core_mask=RKNNLite.NPU_CORE_AUTO
)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

Usage Notes:

- Call load_rknn() first. If no model has been loaded, the API returns -1 directly.
- If a Runtime already exists on the object, the old Runtime is released before reinitialization.
- The API returns -1 if an exception occurs during Runtime creation, model graph construction, Core Mask setup, or version checking.
- inference() can only be called after init_runtime() succeeds. get_sdk_version() also requires the Runtime to be initialized.

3.3 Model Inference

inference

API	inference
Function	Run model inference on the specified inputs and return the output results.
Parameters	inputs : Input data. Type: list ; each item is an ndarray .
	data_type : Input data type. Default: None . The interface lists int8 , uint8 , int16 , float16 , and float32 .
	data_format : Input data layout. Default: None . Currently supports "nhwc" and "nchw" .
	inputs_pass_through : Pass Through flag list for the inputs. Each element uses 0 / 1 to indicate disabled / enabled. Default: None .
	get_frame_id : Whether to obtain the Frame ID. Default: False . Mainly used in asynchronous mode.
	reuse_outputs : Whether to reuse the internal output buffers. Default: False . When True , the API returns views of the internal reusable buffers, suitable for long-running inference loops; note that the next inference overwrites the returned results.
Return Value	Normally returns a list of output ndarray objects. If get_frame_id=True , the Frame ID is appended to the end of the list. Returns None if the Runtime is not initialized or an exception occurs during inference.

Example:

```
outputs = rknn_lite.inference(inputs=[img])
if outputs is None:
    print('Inference failed')
else:
    show_top5(outputs)
```

To reduce memory allocation in a long-running inference loop, enable output reuse:

```
for frame in frames:
    outputs = rknn_lite.inference(
        inputs=[frame],
        reuse_outputs=True
    )
    show_top5(outputs)          # consume immediately; results are overwritten
                                # by the next inference
```

Reference output:

```
-----TOP 5-----
[812]: 0.999442994594574
[404]: 0.0004096269840374589
[657]: 3.284541890025139e-05
[833]: 2.6112385967280716e-05
[895]: 1.8509887013351545e-05
```

Usage Notes:

- `init_runtime()` must complete successfully before this API is called; otherwise, the API returns `None`.
- `inputs`, `data_type`, `data_format`, and `inputs_pass_through` are passed to the Runtime input-setting logic.
- The API returns `None` if an exception occurs while setting inputs, running the model, or retrieving outputs.
- When `get_frame_id=False`, the API returns the normal output list. When `get_frame_id=True`, the Frame ID returned by the Runtime is appended to the end of the output list.
- `reuse_outputs` defaults to `False`, and the returned arrays own their memory and stay valid across calls. Only enable `reuse_outputs=True` when explicitly reducing memory allocation; the returned arrays are then views of the internal reusable buffers and will be overwritten by the next inference, so copy the results first if they must be retained.
- After inference succeeds, perform the required post-processing according to the model type.

3.4 Device, Version, and Platform Queries

list_devices

API	`list_devices`
Function	Enumerate devices currently accessible through ADB and NTB and print the device lists.
Parameters	None
Return Value	`(adb_devices, ntb_devices)` : two `list` objects. Devices that also appear in the NTB list are removed from the returned ADB list.

Example:

```
adb_devices, ntb_devices = rknn_lite.list_devices()
```

get_sdk_version

API	`get_sdk_version`
Function	Get SDK version information.
Parameters	None
Return Value	Normally returns an `sdk_version` string; returns `None` if the Runtime is not initialized or the query fails.

The Runtime must be initialized successfully before calling this API. Because `init_runtime()` requires a model to be loaded, the standard sequence is `load_rknn()` → `init_runtime()` → `get_sdk_version()`.

Example:

```
sdk_version = rknn_lite.get_sdk_version()
if sdk_version is None:
    print('Get SDK version failed')
```

Reference return information:

```
I RKNN: [10:47:23.097] RKNN Runtime Information:
librknnrt version: 1.2.0b1
(a47985372@2021-12-06T10:20:14)

I RKNN: [10:47:23.097] RKNN Driver Information:
version: 0.6.2

I RKNN: [10:47:23.098] RKNN Model Information:
version: 1,
toolkit version: 1.1.2
(compiler version: 1.1.2b13
(1e5726f57@2021-11-29T14:13:55)),
target: RKNPU v2,
target platform: rk3588,
framework name: PyTorch,
framework layout: NCHW
```

list_support_target_platform

API	`list_support_target_platform`
Function	Query the chip platforms on which a given RKNN model can run, or list the currently supported platforms.
Parameters	`rknn_model`: RKNN model path. Default: `None`. When it is `None`, the platform query function is called directly to list the currently supported platforms.
Return Value	Normally returns `support_target`, a `dict`. Returns `None` if an explicitly specified model path does not exist.

Example:

```
support_target = rknn_lite.list_support_target_platform(
    rknn_model='mobilenet_v1.rknn'
)
if support_target is None:
    print('Query target platform failed')
```

Reference result:

```
************************************************
Target platforms filled in RKNN model: ['RK3588']
Target platforms supported by this RKNN model: ['RK3588']
************************************************
```

Usage Notes:

- When `rknn_model=None`, file-existence checking is not performed; the underlying platform query function is called directly.
- When an explicitly specified model path does not exist, the API records an error and returns `None`.

4. Error Handling

The RKNN Toolkit Lite2 Python APIs mainly report execution status through status codes, `None` return values, and Runtime logs. Status-returning APIs should be checked explicitly, and inference and query APIs should be checked for `None` before their results are used.

4.1 Model Loading and Runtime Initialization Errors

`load_rknn()` and `init_runtime()` use the following return values:

- Success: `0`
- Failure: `-1`

Check each initialization step immediately:

```
ret = rknn_lite.load_rknn('./resnet_18.rknn')
if ret != 0:
    print('Load RKNN model failed')
    exit(ret)

ret = rknn_lite.init_runtime(
    core_mask=RKNNLite.NPU_CORE_AUTO
)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

Only continue with `inference()` or `get_sdk_version()` after model loading and Runtime initialization succeed.

4.2 Inference Input and Return-Value Checks

Before calling `inference()`, check that:

- `init_runtime()` has completed successfully;
- `inputs` is a list whose members are the `ndarray` objects required by the model;
- if `data_type`, `data_format`, or `inputs_pass_through` is used, the value matches the actual input;
- in asynchronous mode, set `get_frame_id=True` if frame association is required;
- if `reuse_outputs=True` is enabled, the returned arrays are views of the internal reusable buffers and must not be held across calls; copy them first if the results need to be retained.

`inference()` returns `None` if the Runtime is not initialized, setting inputs fails, model execution fails, or output retrieval fails. Therefore, use a consistent null-value check:

```
outputs = rknn_lite.inference(inputs=[input_data])
if outputs is None:
    print('Inference failed')
    exit(-1)
```

4.3 NPU Core Configuration Checks

The source code notes that `core_mask` is valid only for RK3576 / RK3588. The interface description also marks `RKNNLite.NPU_CORE_0_1_2` as supported only on RK3588.

If manual Core selection is not required, use the default `RKNNLite.NPU_CORE_AUTO`. For a specific chip and Core combination, follow Runtime validation. If Core Mask setup fails, `init_runtime()` returns `-1`.

4.4 Logging

Use `verbose` and `verbose_file` when initializing `RKNNLite` to obtain detailed logs:

```
rknn_lite = RKNNLite(
    verbose=True,
    verbose_file='./inference.log'
)
```

When `verbose=True`, detailed logs are printed to the screen. If `verbose_file` is also specified, the logs are written to the specified file as well.

When model loading, Runtime initialization, or inference fails, use the API return value together with the detailed logs to locate the problem.

5. Appendix

5.1 Reference Documents

RKNN2 SDK Development Guide: RKNN2 software stack, model conversion and quantization, deployment flow, and troubleshooting.

RKNN2 Toolkit Python API Reference: RKNN-Toolkit2 Python API methods, parameters, and return values.

RKNN Toolkit2 documentation:

https://github.com/airockchip/rknn-toolkit2/tree/master/doc

RKNN Toolkit2 project:

https://github.com/airockchip/rknn-toolkit2

5.2 Issue Feedback

Issues can be reported to the Rockchip NPU team through the following channels:

- RKNN QQ group: 1077888690
- GitHub Issue: https://github.com/airockchip/rknn-toolkit2/issues
- Rockchip Redmine: https://redmine.rock-chips.com/

Note: A Redmine account must be enabled through sales or business personnel. If a third-party development board is used, report the issue to the corresponding board vendor first.